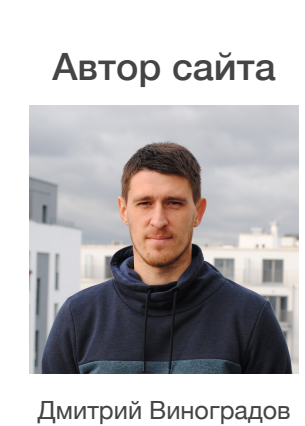
Language

Автор сайта



Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Dagger 2
сделайте код чище
курс от автора сайта
StartAndroid
MADE WITH MYBANNERMAKER.COM

Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid

15 067 подписчиков

Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551360702056

Paypal

Урок 36. User Input. await функции.

18 марта 2024

В этом уроке используем awaitPointerEvent для отлова событий, и рассмотрим прочие await* функции: awaitFirstDown, awaitLongPressOrCancellation, awaitDragOrCancellation, waitForUpOrCancellation, awaitEachGesture.

На прошлом уроке мы узнали про основную функцию awaitPointerEvent, которая возвращает нам данные о касаниях. Теперь попрактикуемся в использовании этой функции для отслеживания простых событий типа click, swipe и long press.

В качестве примера используем код:

```
@Composable
fun HomeScreen() {
   val logs = remember { mutableStateListOf<String>() }
   Column {
       Box(modifier = Modifier
           .size(300.dp)
           .background(Color.LightGray)
           .pointerInput(Unit) {

           }
       )
       Text(
           text = logs.takeLast(12).joinToString("\n"),
           fontSize = 14.sp
       )
   }
}
```

Серый Box с pointerInput, в котором будем кодить. И Text для отображения логов.

click

click представляет собой последовательность из двух событий: Press и Release.

Давайте попробуем пока просто ловить два события подряд, и после этого будем считать, что был клик:

```
.pointerInput(Unit) {
   awaitPointerEventScope {
       while(true) {
           awaitPointerEvent()
           awaitPointerEvent()
           logs.add("click")
       }
   }
}
```

Проверяем:

00:00 00:00

Код работает: показывает нам, что был клик.

Но он же покажет клик и в случае, когда клика не было. Например, если мы решим сделать swipe.

00:00 00:00

Я добавил в логи index для наглядности. Мы двигаем пальцем по экрану - это генерирует кучу Move событий. Через каждые два таких Move мы пишем в лог, что был клик. Это неправильно.

Т.е. просто ловить пару любых событий - не вариант. Нужны события определенного типа. Первое событие должно быть Press, а второе - Release.

Чтобы awaitPointerEvent вернул нужный нам тип события, мы можем поместить его в цикл. Он будет там крутиться и возвращать нам все происходящие события, а мы будем проверять их тип. Как только получим событие нужного нам типа, завершим цикл и пойдем дальше.

Делаем два таких цикла с awaitPointerEvent:

```
.pointerInput(Unit) {
   awaitPointerEventScope {
       while (true) {
           var event1: PointerEvent
           do {
               event1 = awaitPointerEvent()
           } while (event1.type != PointerEventType.Press)
           logs.add("press")

           var event2: PointerEvent
           do {
               event2 = awaitPointerEvent()
           } while (event2.type != PointerEventType.Release)
           logs.add("click")
       }
   }
}
```

В первом цикле ждем появления события Press, а во втором - Release.

Проверяем:

Простые клики работают. Циклы особо не крутятся, т.к. мы сразу получаем нужные нам события.

Попробуем двигать пальцем по экрану.

В момент первого касания мы получаем Press и завершаем первый цикл. А вот второму циклу приходится покрутиться, чтобы отсеять кучу Move и дождаться получения Release. После этого мы уже точно можем сказать, что клик состоялся.

Обратите внимание, что у нас в примере действуют целых три while цикла. Один основной и два вложенных. Вложенные отвечают за фильтр события нужного типа. А основной - за то, чтобы после каждого клика мы возвращались к ожиданию нового клика.

swipe

swipe - по сути тот же клик, только с учетом смещения пальца между Press и Release. Код похож на предыдущий пример:

```
.pointerInput(Unit) {
   awaitPointerEventScope {
       while (true) {
           var event1: PointerEvent
           do {
               event1 = awaitPointerEvent()
           } while (event1.type != PointerEventType.Press)
           val pressPosition = event1.changes.first().position

           var event2: PointerEvent
           do {
               event2 = awaitPointerEvent()
           } while (event2.type != PointerEventType.Release)
           val releasePosition = event2.changes.first().position

           val distance = releasePosition.x - pressPosition.x
           if (distance > 200) {
               logs.add("swipe ${releasePosition.x - pressPosition.x}")
           }
       }
   }
}
```

У нас те же циклы для получения событий Press и Release. Но дополнительно мы сохраняем данные о координатах касания в момент этих событий.

После Release мы вычисляем расстояние по оси X и получаем дистанцию, на которую был сдвинут палец по горизонтали. Если дистанция больше чем 200, считаем, что это был swipe.

Проверяем:

Пример выдает нам swipe, если движение было достаточно продолжительным.

Long press

Long press - это снова комбинация Press + Release. Но в этот раз нам нужно убедиться, что мы НЕ получили Release за определенный промежуток времени. Я для примера возьму промежуток - 2 секунды. Если в течение двух секунд после Release мы получили Release, то long press не состоялся.

Для ожидания чего-либо с таймаутом мы можем использовать функцию withTimeoutOrNull:

```
.pointerInput(Unit) {
   awaitPointerEventScope {
       while (true) {
           var event1: PointerEvent?
           do {
               event1 = awaitPointerEvent()
           } while (event1?.type != PointerEventType.Press)
           logs.add("pressed")

           withTimeoutOrNull(2000) {
               var event2: PointerEvent
               do {
                   event2 = awaitPointerEvent()
               } while (event2.type != PointerEventType.Release)
               logs.add("released")
               event1 = null
           }

           if (event1 != null) {
               logs.add("long press")
           }
       }
   }
}
```

В event1 мы ловим событие Press.

Цикл ожидания Release помещаем в withTimeoutOrNull. Это дает нам два варианта развития событий.

Если в течение двух секунд мы получаем Release, то обнуляем Press. Для нас это будет означать, что long press не состоялся.

Если за две секунды Release так и не пришел, то awaitPointerEvent внутри withTimeoutOrNull выбросит специальный Exception и, тем самым, прервет выполнение блока кода в withTimeoutOrNull. А основной код пойдет дальше.

Далее мы проверяем event1.

Если он null, значит ранее мы получили Release меньше чем за 2 секунды, т.е. был обычный клик, а не long press.

Если event1 - не null, значит мы не дождались Release внутри withTimeoutOrNull(2000), т.е. был long press.

Проверяем

Логика в примере простая и не учитывает, например, сдвиг пальца во время удержания. Т.е. если палец двигался (не в рамках погрешности), то вряд ли это уже можно считать long press. Если есть желание, то попробуйте добавить в пример такую проверку.

awaitFirstDown

Кроме основной функции awaitPointerEvent нам доступно еще несколько await* функций. Под капотом все они используют awaitPointerEvent и, по сути, являются просто обертками для выполнения определенных операций, похожих на то, что мы только что делали.

awaitFirstDown - ждет первого касания. Мы можем его использовать в наших предыдущих примерах вместо первого цикла, в котором мы ловили Press событие.

Перепишем пример, где мы ловим клик:

```
.pointerInput(Unit) {
   awaitPointerEventScope {
       while (true) {
           awaitFirstDown()
           logs.add("pressed")

           var event2: PointerEvent
           do {
               event2 = awaitPointerEvent()
           } while (event2.type != PointerEventType.Release)
           logs.add("click")
       }
   }
}
```

Вместо первого цикла у нас теперь работает awaitFirstDown.

Посмотрим на его исходник:

```
suspend fun AwaitPointerEventScope.awaitFirstDown(
   requireUnconsumed: Boolean = true,
   pass: PointerEventPass = PointerEventPass.Main,
): PointerInputChange {
   var event: PointerEvent
   do {
       event = awaitPointerEvent(pass)
   } while (
       !event.changes.fastAll {
           if (requireUnconsumed) it.changedToDown() else it.changedToDownIgnoreConsumed()
       }
   )
   return event.changes[0]
}
```

Под капотом видим тот же цикл с awaitPointerEvent, только условие более продвинутое, чем наша простая проверка на тип Press.

Функция возвращает данные (PointerInputChange) первого касания.

awaitLongPressOrCancellation

awaitLongPressOrCancellation - функция получения long press события

```
.pointerInput(Unit) {
   awaitPointerEventScope {
       while (true) {
           val firstDown = awaitFirstDown()
           val longPress = awaitLongPressOrCancellation(firstDown.id)
           logs.add("long press: ${longPress != null}")
       }
   }
}
```

На вход она ожидает PointerId касания. Передаем ему тот, которые получили от awaitFirstDown.

Если long press случился, мы получим PointerInputChange, иначе null.

Проверяем

Исходник приводить не буду, он гораздо сложнее того, что мы делали чуть ранее в этом уроке. Но он тоже использует функцию с таймаутом - withTimeout.

Также эта функция определяет, что касание вышло за границу Box. Поэтому в его названии есть слово Cancellation.

Касание может быть Cancelled и по другой причине, не только при выходе за рамки Box. Об этом еще поговорим в следующих уроках.

awaitDragOrCancellation

awaitDragOrCancellation - возвращает нам PointerInputChange данные о перемещении пальца (drag), пока касание активно. Если касание прекратилось или Cancelled, то функция вернет null.

```
.pointerInput(Unit) {
   awaitPointerEventScope {
       while (true) {
           val press = awaitFirstDown()
           var drag: PointerInputChange?
           do {
               drag = awaitDragOrCancellation(press.id)
               logs.add("drag ${drag?.position?.x}")
           } while (drag != null)
           logs.add("released")
       }
   }
}
```

Крутим его в цикле, пока он возвращает не null.

Исходники там тоже не самые простые, приводить их не буду. Но в итоге все снова сводится к циклу с awaitPointerEvent. Плюс проверки, что касание еще активно и что был сдвиг.

Проверяем:

Есть две дополнительные версии этой функции: awaitVerticalDragOrCancellation и awaitHorizontalDragOrCancellation. Под капотом там все то же самое, но они возвращают новый PointerInputChange только в случае, если сдвиг был по вертикали/горизонтали

Есть более короткая версия этой функции - drag. Она прячет цикл под капот, а данные мы получаем в колбэк.

```
.pointerInput(Unit) {
   awaitPointerEventScope {
       while (true) {
           val press = awaitFirstDown()
           drag(press.id) {
               logs.add("drag ${it.position.x.toInt() - press.position.x.toInt()}")
           }
           logs.add("released")
       }
   }
}
```

В лог выводим расстояние по горизонтали от начальной точки касания до текущей.

Проверяем:

Для drag также есть дополнительные версии: verticalDrag и horizontalDrag.

waitForUpOrCancellation

waitForUpOrCancellation - это замена нашему второму циклу, в котором мы ловили Release.

Он вернет последний PointerInputChange, если касаний больше нет. Или null - если касание Cancelled.

```
.pointerInput(Unit) {
   awaitPointerEventScope {
       while (true) {
           awaitFirstDown()
           logs.add("press")
           val lastChange = waitForUpOrCancellation()
           logs.add("click: ${lastChange != null}")
       }
   }
}
```

Проверяем:

Под капотом тут снова цикл с awaitPointerEvent и проверкой, что все касания завершились.

```
while (true) {
   val event = awaitPointerEvent(pass)
   if (event.changes.fastAll { it.changedToUp() }) {
       // All pointers are up
       return event.changes[0]
   }
   …
}
```

awaitEachGesture

Ранее я упоминал, что у нас в awaitPointerEventScope есть основной цикл и несколько вложенных. Вложенные циклы крутят в себе awaitPointerEvent, пока не получат нужное событие. А основной отвечает за то, чтобы по окончанию одного жеста (клика, свайпа) начать ждать следующий.

Мы только что рассмотрели несколько функций, которые убирают под капот создание вложенных циклов. В итоге наш код становится лаконичнее и правильнее с точки зрения обработки различных edge-кейсов.

Для основного цикла тоже есть такая функция - awaitEachGesture. Причем она является оберткой над awaitPointerEventScope. Т.е. мы вызываем ее напрямую в pointerInput:

```
.pointerInput(Unit) {
   awaitEachGesture {
       // code
   }
}
```

Под капотом это равно примерно следующему:

```
.pointerInput(Unit) {
   awaitPointerEventScope {
       while (true) {
           // code
           waitForUpOrCancellation()
       }
   }
}
```

Т.е. awaitEachGesture запускает awaitPointerEventScope с основным циклом. Но тут важно понимать, когда цикл пойдет в начало следующей итерации.

Рассмотрим пример:

```
.pointerInput(Unit) {
   awaitEachGesture {
       val event = awaitPointerEvent()
       logs.add("event = ${event.type}")
   }
}
```

Если мы поставим палец и начнем двигать, то кажется, что в лог сначала придет Press, а затем посыпется куча Move. Но это не совсем так.

Когда мы поставим палец, awaitPointerEvent вернет нам Press, и мы отправим его в лог. На этом наш блок кода завершен. Но для awaitEachGesture этого недостаточно, чтобы начать выполнение этого блока с начала и ждать новое событие. Для него текущая итерация цикла завершится, когда он дождется waitForUpOrCancellation. Поэтому все наши Move просто игнорируются.

А вот когда мы уберем палец, то для awaitEachGesture это будет сигналом, что этот жест завершен, и можно запускать новую итерацию для ожидания следующего жеста.

Т.е. awaitEachGesture начнет новую итерацию цикла, когда предыдущая итерация завершена И все касания завершены.

Это, впрочем, не мешает нам самим использовать waitForUpOrCancellation. Пример с кликом теперь выглядит так:

```
.pointerInput(Unit) {
   awaitEachGesture {
       awaitFirstDown()
       logs.add("press")
       val lastChange = waitForUpOrCancellation()
       logs.add("click: ${lastChange != null}")
   }
}
```

А в начале урока он выглядел так:

```
.pointerInput(Unit) {
   awaitPointerEventScope {
       while (true) {
           var event1: PointerEvent
           do {
               event1 = awaitPointerEvent()
           } while (event1.type != PointerEventType.Press)
           logs.add("press")

           var event2: PointerEvent
           do {
               event2 = awaitPointerEvent()
           } while (event2.type != PointerEventType.Release)
           logs.add("click")
       }
   }
}
```

currentEvent

В конце урока имеет смысл упомянуть о полезном поле - currentEvent.

Мы можем использовать его в любой момент, чтобы получить информацию о последнем произошедшем событии.

```
.pointerInput(Unit) {
   awaitEachGesture {
       val firstDown = awaitFirstDown()
       logs.add("${currentEvent.type}")
       drag(firstDown.id) {
           logs.add("${currentEvent.type}")
       }
       logs.add("${currentEvent.type}")
   }
}
```

Результат:

Присоединяйтесь к нам в **Telegram**:

- в канале StartAndroid публикуются ссылки на новые статьи с сайта startandroid.ru и интересные материалы с хабра, medium.com и т.п.

- в чатах решаем возникающие вопросы и проблемы по различным темам: Android, Compose, Kotlin, RxJava, Dagger, Тестирование, Performance

- ну и если просто хочется поговорить с коллегами по разработке, то есть чат Флудильня

Назад Вперёд

Комментарии

# **RE: Урок 36. User Input. await функции.** — **Игорь** 25.03.2024 11:17
В разделе "Long press": Если в течение двух секунд после Release (здесь видимо опечатка и должно быть Press) мы получили Release, то long press не состоялся.
Ответить

# **Видео фрагменты** — **Владимир** 10.04.2024 13:07
Почему-то видео не воспроизводится, на предыдущем уроке все было ок
Ответить

Обновить список комментариев
RSS лента комментариев этой записи

Добавить комментарий

Тема

Отправить

JComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.